# Elements of AIML Lab
# Experiment 09 (After-submission Reference): Imbalanced Classification

Tofik Ali

March 1, 2026

## Contents

**1 How to use this reference** **1**

**2 The core problem** **1**

**3 Worked example (self-contained)** **2**
3.1 Step 1: create imbalanced data . . . . . 2
3.2 Step 2: baseline model + evaluation . . . . . 2
3.3 Step 3A: class weights (simple and effective) . . . . . 2
3.4 Step 3B: resampling (train only) . . . . . 2
3.5 Step 4 (optional): probability scores + PR-AUC . . . . . 3

**4 What strong observations look like** **3**

**5 Troubleshooting** **3**

## 1 How to use this reference

This reference is meant to be shared **after you submit** Experiment 09. It explains why accuracy fails on imbalanced data and shows standard improvement methods with correct evaluation practice.

## 2 The core problem

If positives are rare (e.g., 1% fraud), a model can get 99% accuracy by predicting "not fraud" always. Therefore, for rare positives we usually focus on:

- recall (catching positives),
- precision (avoiding false alarms),
- F1, PR-AUC (optional but strong),
- confusion matrix (always).

# 3 Worked example (self-contained)

If you used a real dataset, your numbers differ. This example creates an imbalanced dataset to demonstrate the workflow.

## 3.1 Step 1: create imbalanced data

```
import numpy as np
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split

X, y = make_classification(
    n_samples=4000,
    n_features=10,
    n_informative=4,
    n_redundant=2,
    weights=[0.97, 0.03], # 3% positive class
    random_state=42,
)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

## 3.2 Step 2: baseline model + evaluation

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, classification_report

base = LogisticRegression(max_iter=2000)
base.fit(X_train, y_train)
pred = base.predict(X_test)

print(confusion_matrix(y_test, pred))
print(classification_report(y_test, pred, digits=3))
```

Interpret the confusion matrix: if FN is high, the model is missing positives.

## 3.3 Step 3A: class weights (simple and effective)

```
weighted = LogisticRegression(max_iter=2000, class_weight="balanced")
weighted.fit(X_train, y_train)
pred_w = weighted.predict(X_test)

print(confusion_matrix(y_test, pred_w))
print(classification_report(y_test, pred_w, digits=3))
```

Typical outcome: recall increases, precision may decrease (more false alarms). Explain the trade-off.

## 3.4 Step 3B: resampling (train only)

If you have `imbalanced-learn` installed:

```
from imblearn.over_sampling import SMOTE
```

```
sm = SMOTE(random_state=42)
X_res, y_res = sm.fit_resample(X_train, y_train)
print("Before:", np.bincount(y_train), "After:", np.bincount(y_res))
```

Then train on `X_res, y_res` and evaluate on the original test set. **Rule:** never resample before the train/test split.

### 3.5 Step 4 (optional): probability scores + PR-AUC

```
from sklearn.metrics import average_precision_score

scores = weighted.predict_proba(X_test)[:, 1]
ap = average_precision_score(y_test, scores)
print("PR-AUC (Average Precision):", round(ap, 4))
```

## 4 What strong observations look like

- "Baseline accuracy was high, but recall for the minority class was near zero, so it missed most positives."
- "Using class_weight increased recall from 0.10 to 0.72 with a moderate precision drop; this is acceptable when FN is costly."
- "Resampling improved recall further but increased false positives; threshold tuning may be needed depending on domain costs."

## 5 Troubleshooting

- **No `predict_proba`:** some models require probability-enabled settings.
- **Scores look perfect:** check for leakage (resampling before split, or target leakage features).
- **Unstable results:** use stratified splits and fix `random_state`.

*Sources:* Provost and Fawcett, *Data Science for Business.*;McKinney, *Python for Data Analysis.*;WEKA Documentation (Waikato Environment for Knowledge Analysis), accessed 2026-02-28.;Matplotlib and Seaborn documentation, accessed 2026-02-28.